ARQUITECTURA DEL SISTEMA

Sistema de Gestión de Estacionamiento

Tecnología: Java | Diseño de Arquitectura del Sistema

Rol	Arquitecto de Solución
Dirigido a	Desarrollador
Framework	Microsoft Solutions Framework (MSF)
Metodología	Modelo Iterativo
Tecnología	Java
Estructura	6 clases principales + relaciones UML + documentación

Proyecto Universitario

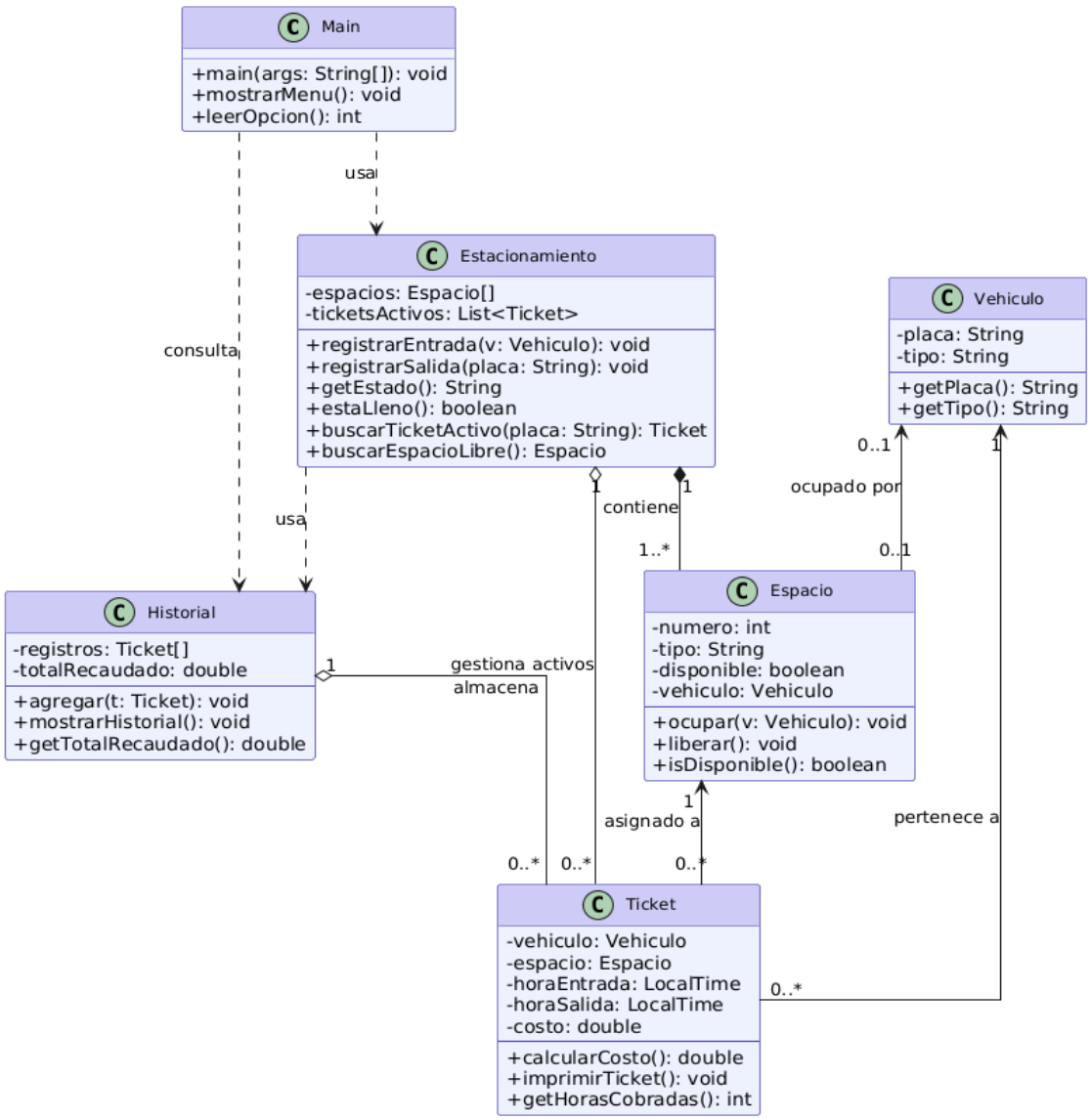
1. Objetivo de la arquitectura

La arquitectura del sistema define la estructura general de la solución en Java, estableciendo las clases principales, sus atributos, métodos y relaciones, con el fin de garantizar que el sistema sea ordenado, entendible y fácil de implementar. Esta arquitectura se alinea con el objetivo del proyecto: gestionar la entrada y salida de vehículos, calcular cobros automáticamente y consultar el estado e historial del estacionamiento.

2. Diagrama de clases

El diagrama de clases representa la estructura estática del sistema. En él se identifican las clases principales del proyecto, sus atributos, métodos y relaciones. El sistema está compuesto por seis clases principales: ***Main***, ***Estacionamiento***, ***Vehículo***, ***Espacio***, ***Ticket*** e ***Historial***, tal como se define en la arquitectura base del proyecto.

2.1 Diagrama de clases



2.2 Descripción breve del diagrama

El diagrama muestra que ***Main*** utiliza a ***Estacionamiento*** como clase principal de control. ***Estacionamiento*** administra los espacios disponibles y los tickets activos. ***Espacio*** puede estar ocupado por un ***Vehículo***, mientras que ***Ticket*** registra la relación entre el vehículo, el espacio asignado, la hora de entrada, la hora de salida y el costo. Finalmente, ***Historial*** almacena los tickets cerrados y el total recaudado.

3. Relaciones entre clases

Las relaciones definidas en la arquitectura permiten organizar correctamente las responsabilidades de cada clase y evitar mezclar funciones dentro del sistema.

3.1 Main → Estacionamiento

La clase ***Main*** usa a ***Estacionamiento*** para ejecutar las operaciones principales del sistema, como registrar entrada, registrar salida y consultar el estado. ***Main*** actúa como punto de inicio y control del flujo del programa.

3.2 Main → Historial

La clase ***Main*** también consulta a ***Historial*** cuando se requiere mostrar registros del día o el total recaudado.

3.3 Estacionamiento → Espacio

Estacionamiento contiene varios objetos ***Espacio***, ya que administra todos los lugares disponibles del parqueadero. Esta relación es fundamental para controlar ocupación, disponibilidad y asignación.

3.4 Estacionamiento → Ticket

Estacionamiento gestiona los tickets activos, es decir, aquellos vehículos que todavía no han registrado salida. Por eso mantiene una colección de objetos ***Ticket***.

3.5 Espacio → Vehículo

Un ***Espacio*** puede estar ocupado por un ***Vehículo*** o puede estar libre. Esta relación permite saber qué vehículo está en qué lugar.

3.6 Ticket → Vehículo

Cada ***Ticket*** pertenece a un ***Vehículo***, ya que registra la entrada y salida de un vehículo específico.

3.7 Ticket → Espacio

Cada ***Ticket*** también se asocia con un ***Espacio***, porque el sistema debe guardar cuál fue el lugar asignado durante la estancia del vehículo.

3.8 Historial → Ticket

Historial almacena los tickets finalizados, permitiendo consultar movimientos anteriores y calcular el total recaudado del día.

4. Documentación de la arquitectura

4.1 Clases principales del sistema

De acuerdo con la estructura del proyecto, las clases principales y sus responsabilidades son las siguientes:

Main

Es la clase de inicio del sistema. Su función es mostrar el menú, leer opciones e invocar las operaciones principales del sistema.

Estacionamiento

Es la clase central de la lógica del sistema. Administra los espacios del estacionamiento, registra entradas y salidas, controla los tickets activos y devuelve el estado actual del parqueadero.

Vehículo

Representa a cada vehículo que ingresa al estacionamiento. Contiene datos básicos como placa, tipo y hora de entrada.

Espacio

Representa un espacio físico del estacionamiento. Tiene número, tipo, estado de disponibilidad y referencia al vehículo que lo ocupa.

Ticket

Representa el registro de permanencia de un vehículo dentro del estacionamiento. Guarda vehículo, espacio, hora de entrada, hora de salida y costo. También se encarga del cálculo de cobro.

Historial

Guarda los tickets cerrados y lleva el control del total recaudado. Permite consultar el historial del día.

4.2 Reglas del negocio consideradas en la arquitectura

La arquitectura fue diseñada tomando en cuenta las reglas del negocio establecidas en el proyecto: no permitir ingreso si el estacionamiento está lleno, no permitir placas duplicadas activas, mostrar error si se intenta registrar la salida de una placa inexistente, aplicar un cobro mínimo de una hora y redondear hacia arriba las fracciones de hora. Estas validaciones son gestionadas principalmente por las clases ***Estacionamiento*** (control de ingreso y capacidad), ***Ticket*** (cálculo de costo y tiempo) e ***Historial*** (acumulación de ingresos).

4.3 Decisiones de diseño

Se decidió separar las responsabilidades del sistema por clases para mantener una arquitectura clara y modular. ***Main*** solo coordina la ejecución; ***Estacionamiento*** concentra la lógica principal; ***Vehículo*** y ***Espacio*** representan entidades del dominio; ***Ticket*** encapsula el control del cobro; e ***Historial*** se encarga del almacenamiento de registros cerrados. Esta separación facilita el mantenimiento, las pruebas y futuras mejoras del sistema.

4.4 Estructura del proyecto

La arquitectura también considera la organización general del proyecto en carpetas, incluyendo código fuente, documentación, pruebas y entregable ejecutable. La estructura propuesta contempla ***/src/***, ***/docs/***, ***/test/***, ***/release/*** y el archivo ***README.md***.

4.5 Aporte del Arquitecto de Solución en las iteraciones

Durante la primera iteración, como Arquitecto de Solución diseñe el diagrama de clases, las relaciones y la estructura general del proyecto. En la segunda iteración, revise que el código respete la arquitectura definida y oriente al desarrollador. En la tercera iteración, documente la arquitectura final y verifique que no exista deuda técnica importante.

5. En Conclusión

La arquitectura propuesta organiza correctamente el sistema de gestión de estacionamiento en clases bien definidas, con responsabilidades separadas y relaciones coherentes. Esto permite que el sistema sea comprensible, mantenible y alineado con los requisitos funcionales del proyecto. El diagrama de clases y la documentación arquitectónica evidencian el trabajo realizado por el rol de Arquitecto de Solución dentro del enfoque MSF con modelo iterativo.